PRAKTEK TEKNOLOGI IOT II

LAPORAN PRAKTEK

TEKNOLOGI IOT II

Disusun dalam rangka memenuhi salah satu syarat kelengkapan nilai Praktikum

Mata Kuliah Teknologi IOT II

DISUSUN OLEH :

NAMA : Raffi Maulana Abdillah

NPM 25782021

PRODI : TEKNOLOGI REKAYASA INTERNET

KELAS : 3A

DOSEN : Dr. ir. Septafiansyah Dwi Putra, S. T.,M.T.

: Agus Ambarwari, S.Pd.,M.Kom.

8.5.1 Praktikum 1: Peningkatan Ekosistem IoT Server dengan InfluxDB v2 dan Grafana pada Docker Compose

Pada praktikum ini, kita akan memperluas konfigurasi server IoT lokal yang telah dibuat pada Modul 7 dengan menambahkan kontainer **InfluxDB versi 2** (sebagai basis data deret waktu) dan kontainer **Grafana** (sebagai persiapan antarmuka visualisasi industri untuk Modul 9).

```
+-------------------------------------------------------------------------------+
| STRUKTUR DIREKTORI PROYEK IOT-SERVER (LENGKAP DENGAN INFLUXDB & GRAFANA)      |
+-------------------------------------------------------------------------------+
| iot-server/                                                                   |
| ├── docker-compose.yml      # Konfigurasi orkestrasi 4 layanan terpadu        |
| ├── mosquitto/                                                                |
| │   ├── config/                                                               |
| │   │   ├── mosquitto.conf   # Konfigurasi port 1883, 9001, allow_anonymous false |
| │   │   └── password.txt     # Berkas kata sandi terenkripsi (dari Modul 7)   |
| │   ├── data/              # Volume data persisten Mosquitto                  |
| │   └── log/               # Volume catatan log Mosquitto                     |
| ├── nodered/               # Volume data alur kerja (flows) Node-RED         |
| ├── influxdb/                                                                 |
| │   ├── data/              # Volume penyimpanan data mesin TSM InfluxDB v2    |
| │   └── config/            # Berkas konfigurasi mesin InfluxDB v2             |
| └── grafana/               # Volume penyimpanan dasbor dan preferensi Grafana |
+-------------------------------------------------------------------------------+
```

Langkah Kerja:

1. Buka aplikasi Terminal atau PowerShell di laptop Anda, lalu arahkan (*cd*) ke direktori kerja iot-server milik Anda:

```
cd D:/iot-server
```

2. Buat subdirektori penyimpanan fisik persisten baru untuk InfluxDB dan Grafana:
   - **Pengguna Windows (PowerShell):**

```
New-Item -ItemType Directory -Force -Path influxdb/data, influxdb/config, grafana
```

   - **Pengguna Linux / macOS:**

```
mkdir -p influxdb/data influxdb/config grafana
```

3. Hentikan tumpukan kontainer Modul 7 yang mungkin masih berjalan:

```
docker compose down
```

4. Buka berkas **docker-compose.yml** di dalam folder iot-server menggunakan editor teks (VS Code atau Notepad). Perbarui seluruh isinya dengan dokumen konfigurasi empat layanan terpadu berikut:

```
services:
  # ============================================================
  # 1. Layanan MQTT Broker: Eclipse Mosquitto
  # ============================================================
  mqtt:
    image: eclipse-mosquitto:latest
    container_name: mqtt-broker
    restart: unless-stopped
    ports:
      - "1883:1883"  # Port TCP MQTT standar untuk mikrokontroler NodeMCU
      - "9001:9001"  # Port WebSockets untuk klien web browser
    volumes:
      - ./mosquitto/config:/mosquitto/config
```

```
    - ./mosquitto/data:/mosquitto/data
    - ./mosquitto/log:/mosquitto/log

# ============================================================
# 2. Layanan Orkestrasi & Rule Engine: Node-RED
# ============================================================
nodered:
  image: nodered/node-red:latest
  container_name: node-red
  restart: unless-stopped
  ports:
    - "1880:1880"  # Antarmuka web pengembang Node-RED
  environment:
    TZ: Asia/Jakarta
  volumes:
    - ./nodered:/data
  depends_on:
    - mqtt

# ============================================================
# 3. Layanan Basis Data Deret Waktu: InfluxDB v2
# ============================================================
influxdb:
  image: influxdb:2.7
  container_name: influxdb2
  restart: unless-stopped
  ports:
    - "8086:8086"  # Port HTTP API & Web UI InfluxDB v2
  environment:
    # Inisialisasi otomatis akun administrator dan bucket awal
    DOCKER_INFLUXDB_INIT_MODE: setup
    DOCKER_INFLUXDB_INIT_USERNAME: admin
    DOCKER_INFLUXDB_INIT_PASSWORD: adminpassword
    DOCKER_INFLUXDB_INIT_ORG: my-org
    DOCKER_INFLUXDB_INIT_BUCKET: my-bucket
    DOCKER_INFLUXDB_INIT_RETENTION: 30d
  volumes:
    - ./influxdb/data:/var/lib/influxdb2
    - ./influxdb/config:/etc/influxdb2

# ============================================================
# 4. Layanan Dasbor Visualisasi Industri: Grafana
# ============================================================
grafana:
  image: grafana/grafana:latest
  container_name: grafana
  restart: unless-stopped
  ports:
    - "3000:3000"  # Port Antarmuka Dasbor Web Grafana
  environment:
```

```
    GF_SECURITY_ADMIN_USER: admin
    GF_SECURITY_ADMIN_PASSWORD: adminpassword
    GF_USERS_ALLOW_SIGN_UP: "false"
  volumes:
    - ./grafana:/var/lib/grafana
  depends_on:
    - influxdb
```

5. Simpan berkas docker-compose.yml. Jalankan perintah berikut untuk mengunduh *image* InfluxDB dan Grafana serta menghidupkan seluruh ekosistem kontainer di latar belakang (*detached mode*):

```
docker compose up -d
```

(Catatan: Proses pertama kali memerlukan beberapa saat untuk mengunduh image dari Docker Hub).

```
C:\Users\ASUS\Documents\iot-server>docker compose down
[+] down 3/3
 ✔Container node-red         Removed                                                    4.4s
 ✔Container mqtt-broker      Removed                                                    1.2s
 ✔Network iot-server_default Removed                                                    0.7s

C:\Users\ASUS\Documents\iot-server>docker compose up -d
[+] up 30/30
 ✔Image grafana/grafana:latest Pulled                                                 240.8s
 ✔Image influxdb:2.7           Pulled                                                 178.3s
 ✔Network iot-server_default   Created                                                  0.3s
 ✔Container influxdb2          Started                                                  4.8s
 ✔Container mqtt-broker        Started                                                  2.4s
 ✔Container node-red           Started                                                  3.0s
 ✔Container grafana            Started                                                  5.6s
```

6. **Verifikasi Status Kontainer:**
   Jalankan perintah berikut untuk memastikan keempat kontainer telah berstatus **Up**:

```
docker compose ps
```

```
C:\Users\ASUS\Documents\iot-server>docker compose ps
NAME          IMAGE                     COMMAND                  SERVICE    CREATED          STATUS                    PORTS
grafana       grafana/grafana:latest    "/run.sh"                grafana    51 seconds ago   Up 46 seconds             0.0.0.0:3000-
>3000/tcp, [::]:3000->3000/tcp
influxdb2     influxdb:2.7              "/entrypoint.sh infl…"   influxdb   51 seconds ago   Up 47 seconds             0.0.0.0:8086-
>8086/tcp, [::]:8086->8086/tcp
mqtt-broker   eclipse-mosquitto:latest  "/docker-entrypoint.…"   mqtt       51 seconds ago   Up 49 seconds             0.0.0.0:1883-
>1883/tcp, [::]:1883->1883/tcp, 0.0.0.0:9001->9001/tcp, [::]:9001->9001/tcp
node-red      nodered/node-red:latest   "./entrypoint.sh"        nodered    51 seconds ago   Up 48 seconds (healthy)   0.0.0.0:1880-
>1880/tcp, [::]:1880->1880/tcp

C:\Users\ASUS\Documents\iot-server>
```

Pastikan pada kolom STATUS seluruh kontainer (mqtt-broker, node-red, influxdb2, dan grafana) menunjukkan keterangan Up ... (healthy) atau Up ....

8.5.2 Praktikum 2: Inisialisasi dan Manajemen Kredensial & API Token InfluxDB v2

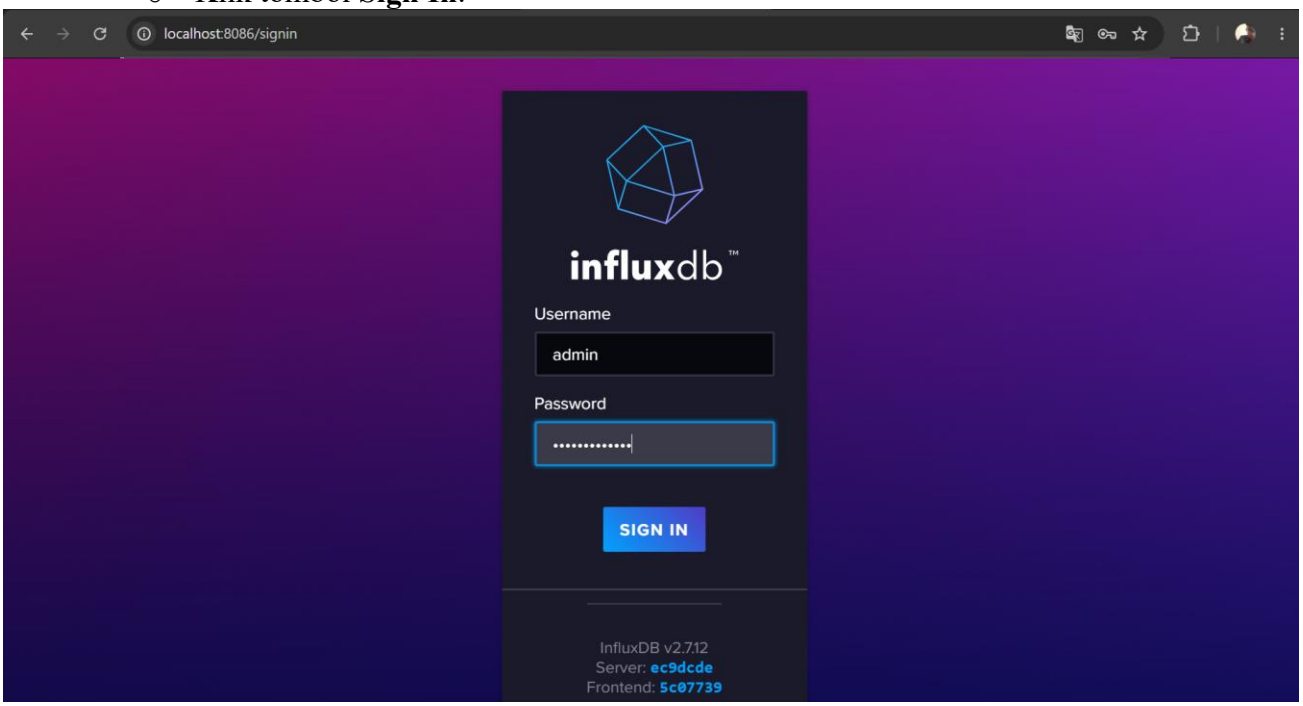
Karena InfluxDB versi 2 mengedepankan keamanan berbasis token, setiap aplikasi luar (seperti Node-RED) wajib menyertakan *API Token* yang valid. Pada praktikum ini, kita akan mengakses antarmuka InfluxDB, memverifikasi organisasi, dan menerbitkan token otorisasi khusus dengan izin Tulis (*Write*).

```
+-------------------------------------------------------------------------------+
| STRATEGI PENERBITAN TOKEN: PRINSIP LEAST PRIVILEGE                            |
+-------------------------------------------------------------------------------+
| [All-Access Admin Token]  ==> DILARANG untuk klien luar (Bisa DROP seluruh DB!)  |
|                                                                               |
| [Custom Write-Only Token] ==> DIBERIKAN ke Node-RED:                          |
```

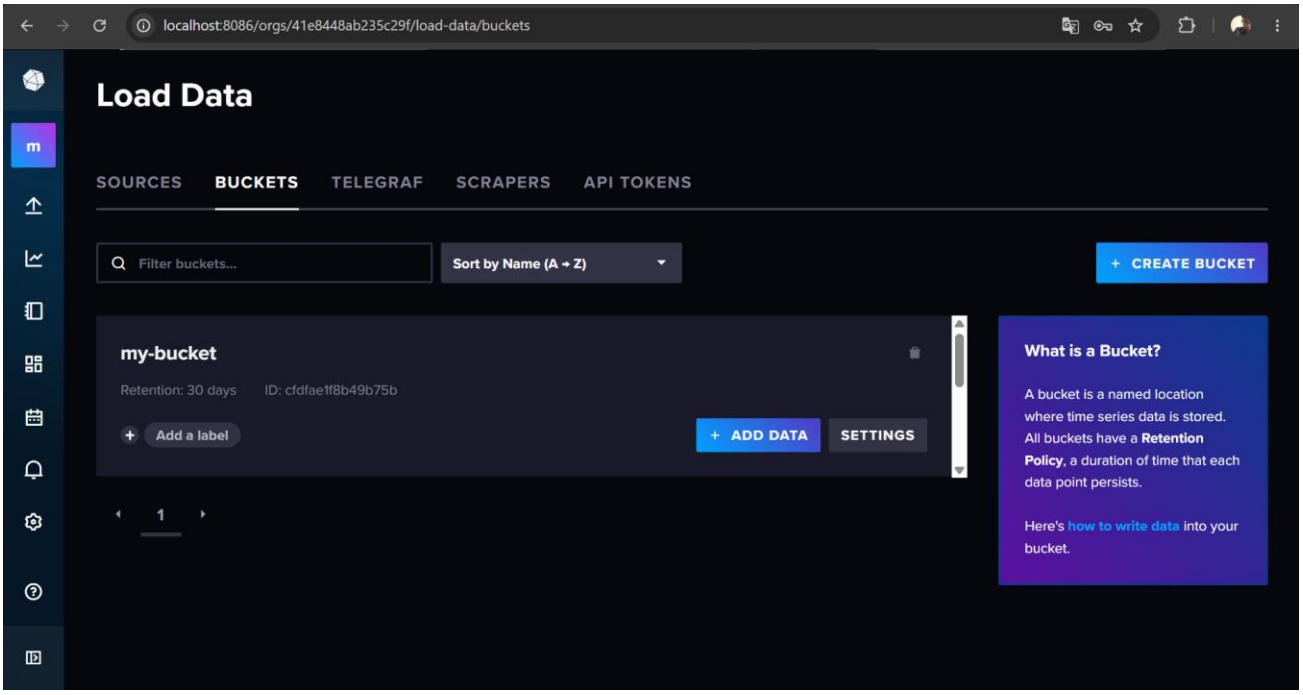
| • Scope: my-bucket |
| • Action: WRITE Only (Aman & Sesuai Standar IIoT) |
+---+

Langkah Kerja:

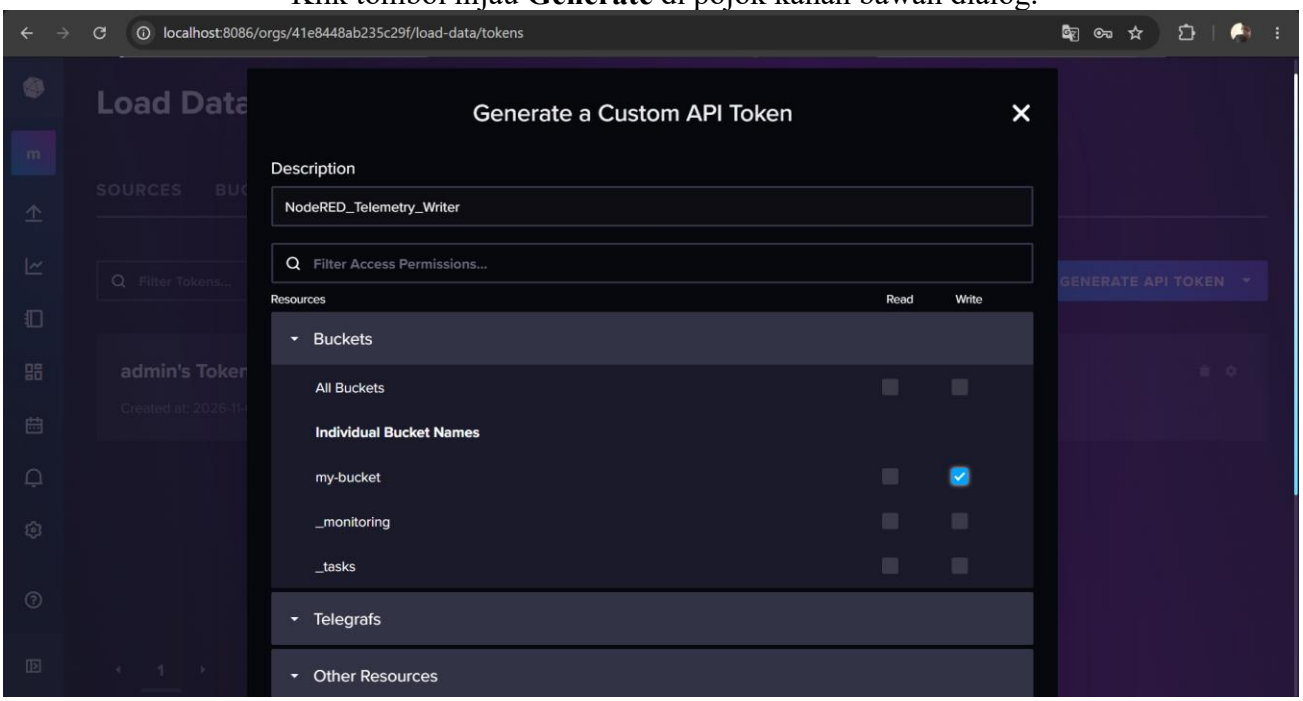
1. Buka peramban web di laptop Anda dan akses alamat: **http://localhost:8086**.
2. Anda akan disajikan halaman masuk (*Sign In*) InfluxDB v2. Masukkan kredensial awal yang telah didefinisikan pada variabel lingkungan docker-compose.yml:
    - **Username:** admin
    - **Password:** adminpassword
    - Klik tombol **Sign In**.



3. Setelah berhasil masuk ke halaman utama (*Get Started*), perhatikan menu vertikal di sisi kiri layer.
4. **Memverifikasi Bucket Penyimpanan:**
    - Klik menu bertanda ikon silinder basis data (**Load Data**) di bilah navigasi kiri, lalu pilih tab **Buckets**.
    - Pastikan bucket bernama **my-bucket** telah tersedia secara otomatis dengan masa retensi (*Retention Period*) 30 hari.



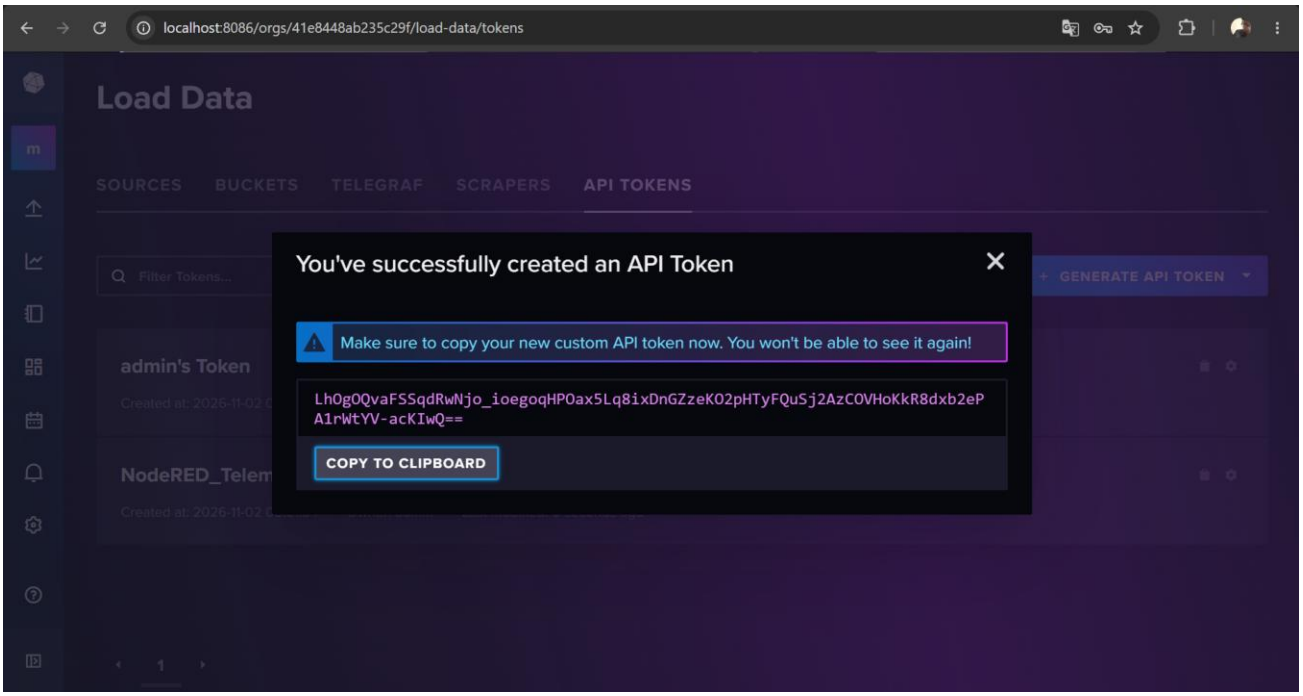
5. **Menerbitkan Token Khusus untuk Node-RED (*Generate Scoped API Token*):**
    - Masih di halaman **Load Data**, klik tab **API Tokens**.
    - Di sebelah kanan atas, klik tombol biru **Generate API Token**, lalu pilih opsi **Custom API Token**.
    - Pada jendela konfigurasi token yang muncul:
        - **Description:** Beri label deskriptif, misalnya: NodeRED_Telemetry_Writer.
        - Pada bagian **Buckets**, cari baris **my-bucket**:
            - Beri tanda centang pada kolom **Write** saja (kosongkan kolom Read untuk menerapkan prinsip *Least Privilege*).
        - Klik tombol hijau **Generate** di pojok kanan bawah dialog.



6. **Menyimpan Token Otorisasi:**
    - Token baru bernama NodeRED_Telemetry_Writer akan muncul pada daftar token.

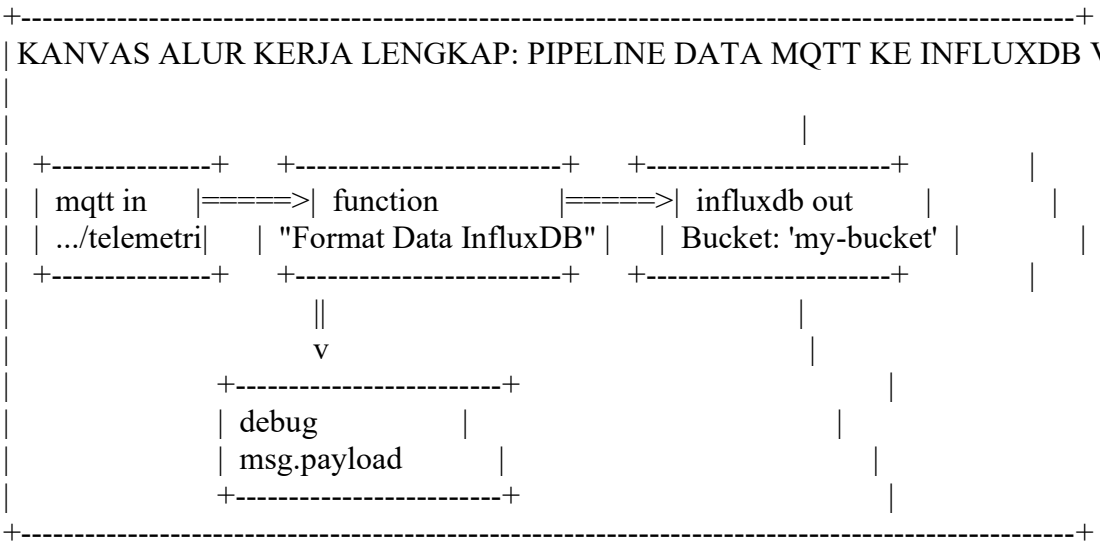
Klik nama token tersebut untuk membuka jendela rincian.

- Klik tombol biru **Copy to Clipboard** untuk menyalin deretan karakter acak panjang tersebut (contoh bentuk token: OqR9kK7h...Wz1Q==).
- Tempelkan (*paste*) token tersebut ke aplikasi teks editor (seperti Notepad) untuk disimpan sementara waktu, karena token ini akan kita gunakan pada konfigurasi Node-RED di Praktikum 3.



8.5.3 Praktikum 3: Instalasi Node InfluxDB dan Perancangan Pipeline Data pada Node-RED

Pada praktikum ini, kita akan menambahkan modul pustaka InfluxDB ke dalam Node-RED, memproses muatan data telemetri MQTT yang dikirimkan oleh NodeMCU ESP8266, mentransformasikannya menjadi struktur array Line Protocol, dan menyimpannya secara otomatis ke basis data InfluxDB.

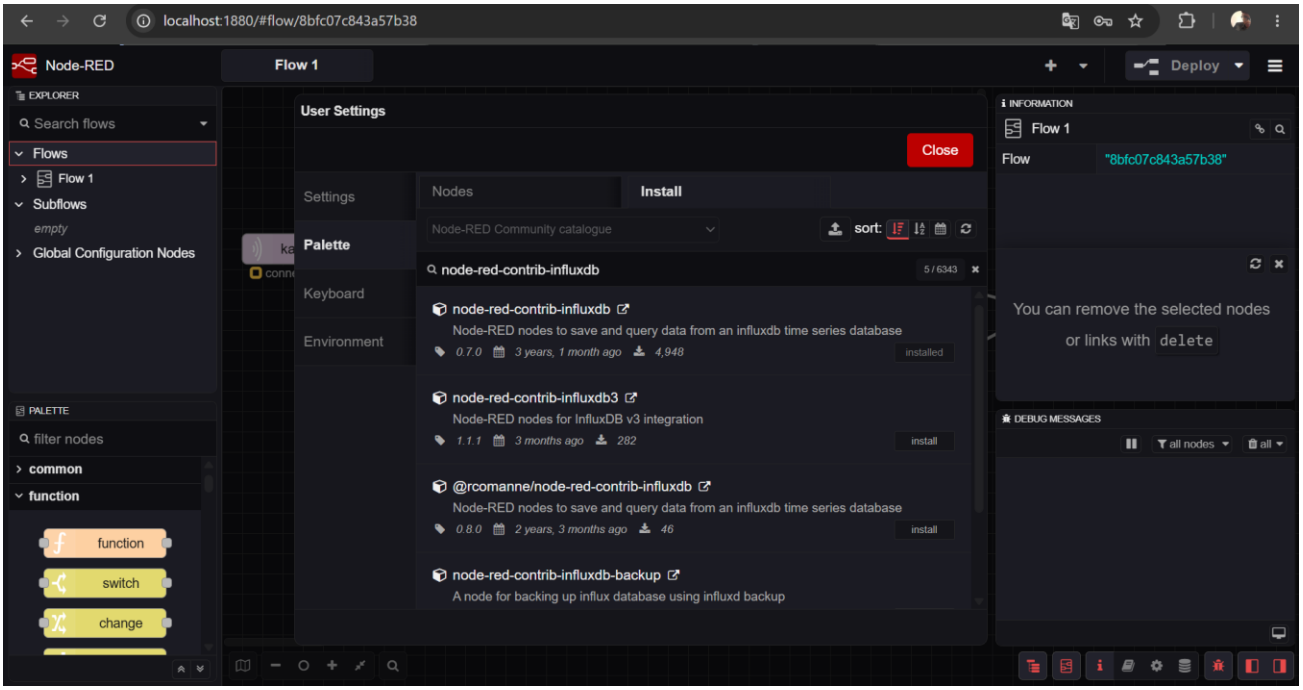


Langkah Kerja:

1. Buka peramban web dan akses antarmuka kerja Node-RED di **http://localhost:1880**.
2. **Memasang Modul node-red-contrib-influxdb:**
   - Klik ikon menu tiga garis horizontal (*Hamburger Menu*) di pojok kanan atas

antarmuka Node-RED, lalu pilih **Manage palette**.

- Buka tab **Install**, kemudian pada kotak pencarian (*search box*), ketik: node-red-contrib-influxdb.
- Pada hasil pencarian modul node-red-contrib-influxdb, klik tombol **install**, lalu konfirmasi dengan menekan **Install** pada dialog pop-up.
- Tunggu sekitar 15-30 detik hingga proses pengunduhan dependensi npm selesai. Setelah berhasil terpasang, sebuah kategori baru bernama **storage** akan muncul di bilah palet kiri dengan simpul influxdb in, influxdb out, dan influxdb batch.
- Klik tombol **Close**.



[TIPS TROUBLESHOOTING JIKA INSTALASI PALETTE TIMEOUT]:
Apabila proses instalasi melalui antarmuka web mengalami kegagalan atau *timeout* akibat koneksi lambat, Anda dapat memasang paket secara langsung di dalam kontainer melalui Terminal laptop:
docker exec -it node-red npm install --save node-red-contrib-influxdb
docker restart node-red

3. **Menyiapkan Alur Data (*Flow Pipeline*):**
   - Di kanvas kerja Node-RED Anda (dapat menggunakan flow kelanjutan dari Modul 7), pastikan node **mqtt in** telah terkonfigurasi mendengarkan topik telemetri:
     - **Server:** Mosquitto Local (mqtt:1883 dengan autentikasi admin/rahasia123)
     - **Topic:** kampus/labiot/telemetri
     - **Output:** a parsed JSON object
4. **Membuat Node Transformasi Format Data (function):**
   - Tarik node **function** dari kategori *function* di bilah palet kiri ke tengah kanvas.
   - Hubungkan port keluaran node mqtt in ke port masukan node function tersebut.
   - Klik ganda node function untuk mengeditnya:
     - **Name:** Format Data InfluxDB
     - Pada tab **Function**, masukkan blok skrip JavaScript terstruktur berikut:
5. //
   ==========
   // TRANSFORMASI DATA TELEMETRI JSON KE STRUKTUR INFLUXDB V2

```
// ============================================================================
// Format baku node-red-contrib-influxdb v2 Out adalah Array dengan 2 Elemen:
// Element 0: Objek FIELDS (Metrik numerik/teks yang diukur, tidak diindeks)
// Element 1: Objek TAGS (Metadata kontekstual string yang diindeks secara cepat)
// ============================================================================

let data = msg.payload;

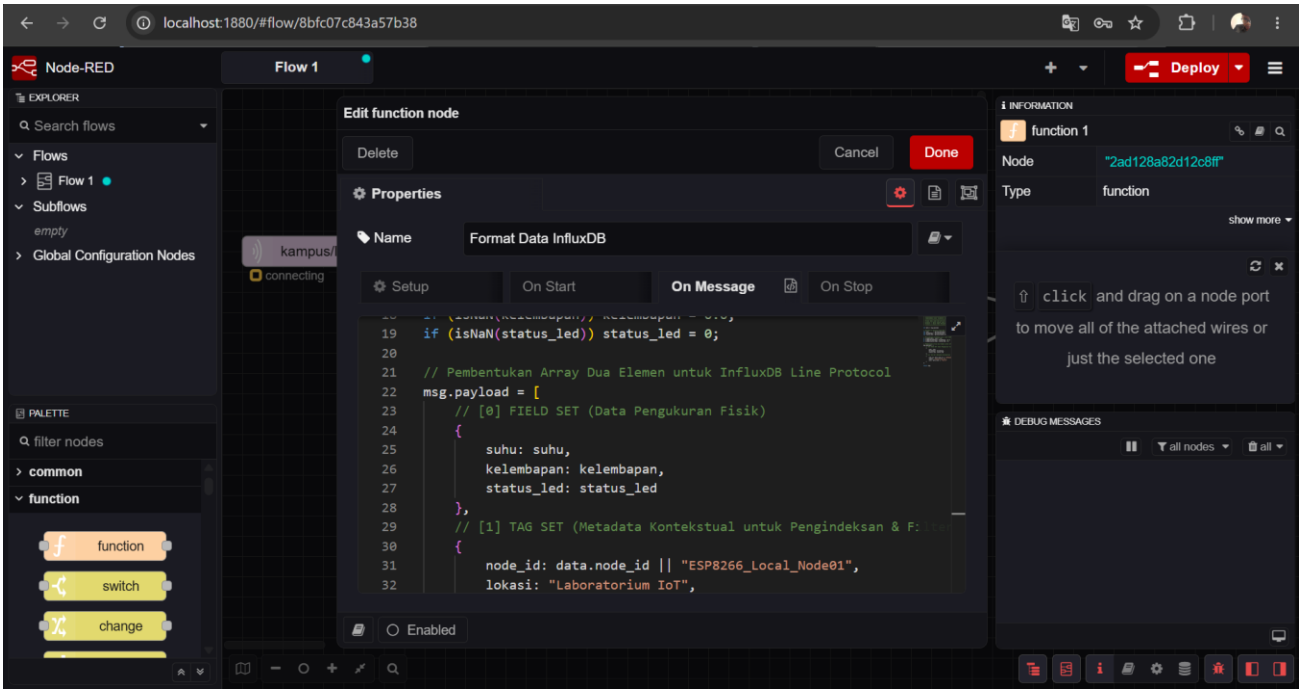
// Validasi & Konversi Tipe Data Numerik Aman
let suhu       = parseFloat(data.suhu);
let kelembapan = parseFloat(data.kelembapan);
let status_led = parseInt(data.status_led);

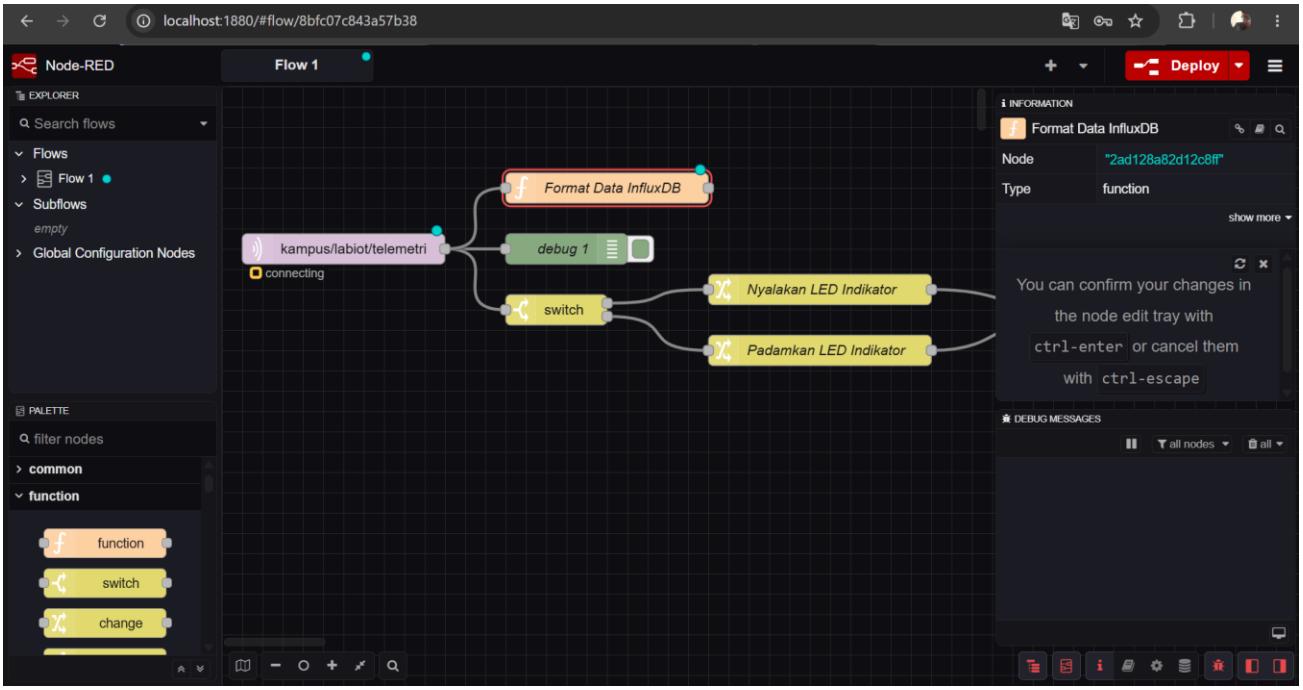
// Perlindungan terhadap nilai rusak (NaN / Not-a-Number)
if (isNaN(suhu)) suhu = 0.0;
if (isNaN(kelembapan)) kelembapan = 0.0;
if (isNaN(status_led)) status_led = 0;

// Pembentukan Array Dua Elemen untuk InfluxDB Line Protocol
msg.payload = [
    // [0] FIELD SET (Data Pengukuran Fisik)
    {
        suhu: suhu,
        kelembapan: kelembapan,
        status_led: status_led
    },
    // [1] TAG SET (Metadata Kontekstual untuk Pengindeksan & Filter Cepat)
    {
        node_id: data.node_id || "ESP8266_Local_Node01",
        lokasi: "Laboratorium IoT",
        zona: data.zona || "Zona-A"
    }
];

return msg;
```

- Klik tombol merah **Done**.



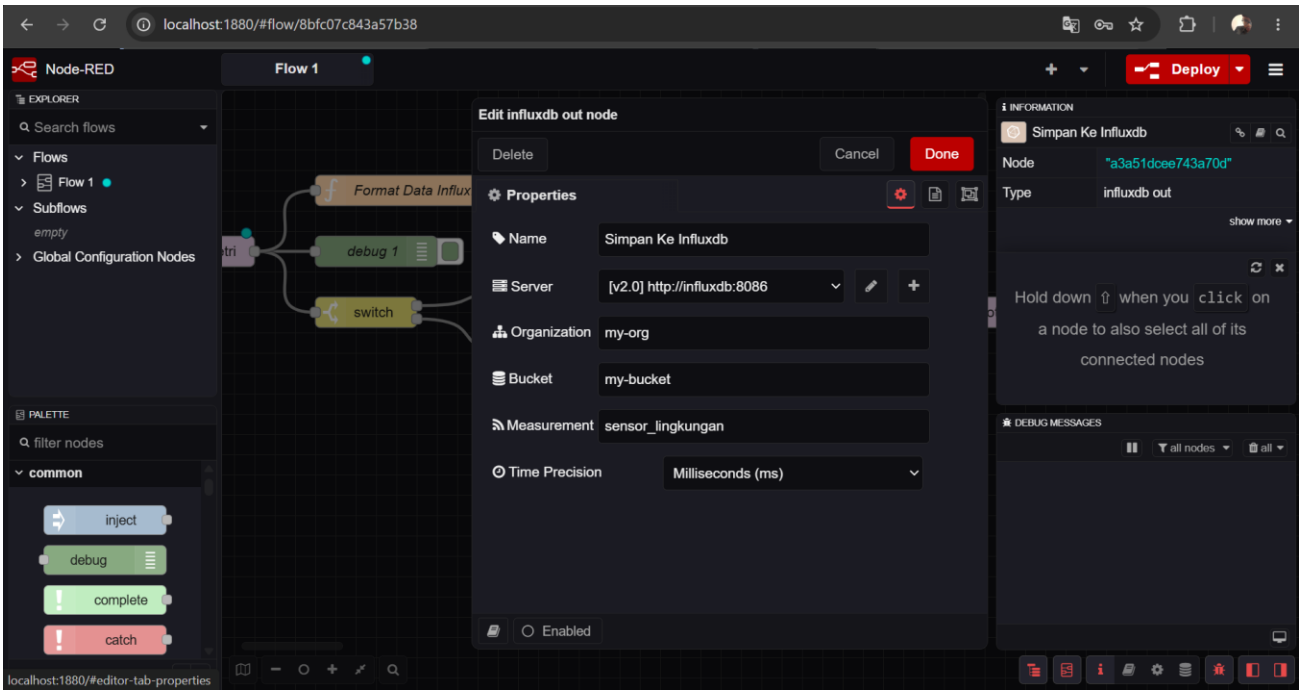


6. **Mengonfigurasi Node Penyimpanan Basis Data (influxdb out):**
   - Seret node **influxdb out** dari kategori *storage* ke kanvas di sebelah kanan node function.
   - Tarik garis penghubung dari keluaran node function ke masukan node influxdb out.
   - Klik ganda node influxdb out untuk membuka panel pengaturannya:
     - Pada baris **Server**, klik ikon pensil ( ) untuk menambahkan konfigurasi koneksi peladen InfluxDB baru:
       - **Version:** Pilih **2.0** *(PENTING: Jangan pilih 1.x!)*.
       - **URL:** Masukkan **http://influxdb:8086**
         (PENTING: Gunakan nama servis Docker influxdb, bukan localhost!

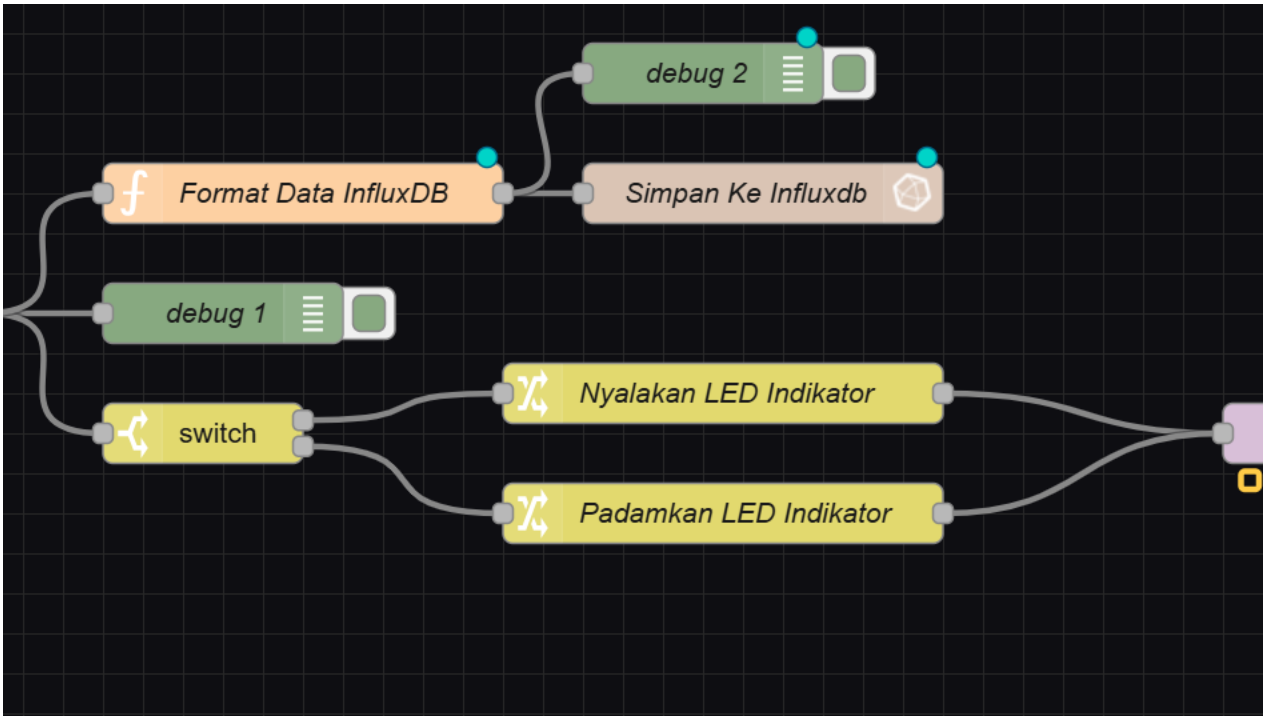
Karena kontainer Node-RED dan InfluxDB berada di jaringan internal Docker yang sama).

- **Token:** Tempelkan (*paste*) deretan karakter API Token yang telah Anda salin pada Praktikum 2 (token NodeRED_Telemetry_Writer).
- Klik tombol merah **Add**.

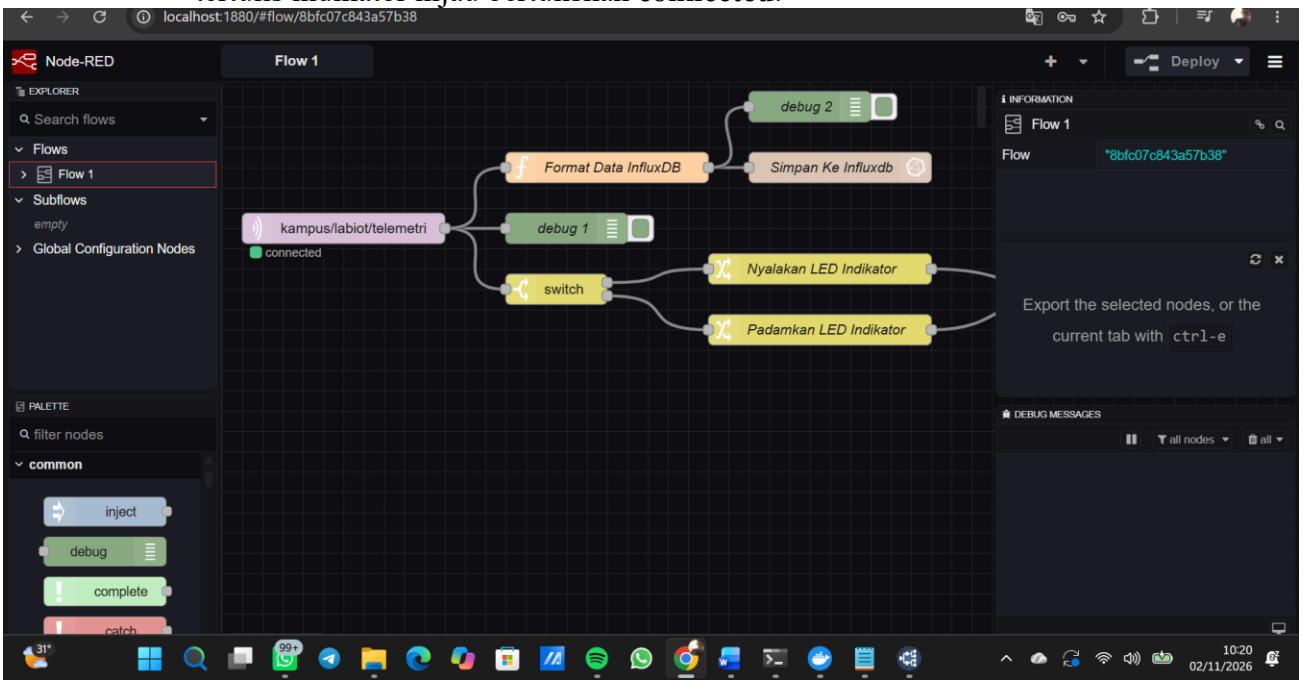
- Pada jendela utama node influxdb out:
  - **Organization:** Masukkan **my-org**
  - **Bucket:** Masukkan **my-bucket**
  - **Measurement:** Masukkan **sensor_lingkungan**
  - **Name:** Beri nama Simpan ke InfluxDB
- Klik tombol merah **Done**.



7. **Menambahkan Node Diagnostik (debug):**
   - Tarik node **debug** dari kategori *common* ke kanvas.
   - Hubungkan keluaran node function ke masukan node debug untuk memantau struktur array data di panel bilah samping (*Debug Sidebar*).



8. **Menerapkan Perubahan (*Deploy*):**
    - Klik tombol merah **Deploy** di sudut kanan atas layar.
    - Pastikan tidak terdapat pesan kesalahan (*syntax error*) dan di bawah node mqtt in tertulis indikator hijau bertuliskan **connected**.



8.5.4 Praktikum 4: Integrasi End-to-End NodeMCU ESP8266 dan Verifikasi Telemetri Time-Series

Pada praktikum ini, kita akan memprogram NodeMCU ESP8266 agar membaca sensor fisik DHT11/22, menyematkan metadata "zona", mengirimkannya ke broker lokal, lalu memverifikasi data yang masuk secara visual melalui antarmuka **Data Explorer** di InfluxDB v2.

Skema Rangkaian Perangkat Keras:

Rangkaian perangkat keras identik dengan Modul 7:

- **Sensor Suhu & Kelembapan (DHT11/DHT22):**
  - Pin VCC →Pin **3V3** NodeMCU
  - Pin GND →Pin **GND** NodeMCU
  - Pin Data →Pin **D4 (GPIO 2)**
- **LED Indikator Aktuator:**
  - Anoda (kaki panjang) →Pin **D1 (GPIO 5)**
  - Katoda (kaki pendek) →Resistor 220 Ω →Pin **GND**

Langkah Pemrograman Mikrokontroler:

1. Buka aplikasi **Arduino IDE**, buat berkas sketsa baru, lalu salin kode program lengkap yang telah diselaraskan dengan arsitektur Modul 7 dan penambahan metadata zona berikut:

```
/*
 *
================================================================================
=======
 * PRAKTIKUM 8: INTEGRASI END-TO-END TELEMETRI SENSOR KE TIME-SERIES
DATABASE
 *
================================================================================
=======
 * Deskripsi:
 * Program ini membaca sensor suhu/kelembapan DHT, menyusun dokumen telemetri JSON
 * yang diperkaya dengan identitas metadata "zona", mengirimkannya ke broker MQTT
 * Mosquitto lokal di laptop secara periodik non-blocking, serta mendengarkan
 * instruksi pengendalian LED secara asinkron.
 *
================================================================================
=======
 */

#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// 1. Kredensial Jaringan Wi-Fi (Sesuaikan dengan SSID & Password Hotspot Anda)
const char* ssid     = "NAMA_WIFI_HOTSPOT";
const char* password = "PASSWORD_HOTSPOT";

// 2. Konfigurasi Broker MQTT Lokal
// Ganti dengan Alamat IPv4 Laptop Anda yang diperoleh melalui perintah 'ipconfig'!
const char* mqttServer = "192.168.1.15";
const int   mqttPort   = 1883;

// Kredensial Autentikasi Broker MQTT (Dikonfigurasi pada Modul 7)
const char* mqttUser   = "admin";
const char* mqttPass   = "rahasia123";

// 3. Konfigurasi Identitas Node & Topik MQTT
```

```
const char* clientId      = "ESP8266_Local_Node01";
const char* topicPubTele  = "kampus/labiot/telemetri";
const char* topicSubCmd   = "kampus/labiot/kendali";

// 4. Konfigurasi Alokasi Pin Perangkat Keras
const byte dhtPin = 2; // Pin D4 (GPIO 2)
const byte ledPin = 5; // Pin D1 (GPIO 5)

#define DHTTYPE DHT11  // Ubah ke DHT22 jika fisik sensor berwarna putih

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

// Variabel Pewaktu Asinkron (Non-Blocking Timer)
unsigned long lastTeleTime = 0;
const unsigned long teleInterval = 4000; // Siklus kirim data telemetri setiap 4 detik

// Penanganan Paket Masuk (Callback Kendali Aktuator)
void onMessageCallback(char* topic, byte* payload, unsigned int length) {
  char incomingBuffer[length + 1];
  memcpy(incomingBuffer, payload, length);
  incomingBuffer[length] = '\0';

  Serial.println("\n---------------------------------------");
  Serial.printf("[KENDALI MASUK] Topik: %s\n", topic);
  Serial.printf("[PAYLOAD]       : %s\n", incomingBuffer);

  // Parsing JSON Instruksi Kendali: {"led": 1} atau {"led": 0}
  StaticJsonDocument<256> doc;
  DeserializationError err = deserializeJson(doc, incomingBuffer);

  if (err) {
    Serial.printf("[ERROR] Gagal parsing JSON instruksi: %s\n", err.c_str());
    return;
  }

  if (doc.containsKey("led")) {
    int stateLED = doc["led"];
    digitalWrite(ledPin, stateLED == 1 ? HIGH : LOW);
    Serial.printf("--> [AKTUATOR] Status LED diubah menjadi: %s\n", stateLED == 1 ?
"MENYALA" : "PADAM");
  }
  Serial.println("---------------------------------------");
}

void connectToWiFi() {
  Serial.println();
  Serial.print("[Wi-Fi] Menghubungkan ke ");
  Serial.println(ssid);
```

```
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\n[Wi-Fi] Berhasil Terhubung!");
  Serial.print("[Wi-Fi] IP NodeMCU: ");
  Serial.println(WiFi.localIP());
}

void reconnectMQTT() {
  while (!client.connected()) {
    Serial.print("[MQTT] Menghubungkan ke Broker Lokal (");
    Serial.print(mqttServer);
    Serial.print(")...");

    // Menghubungkan dengan kredensial resmi
    if (client.connect(clientId, mqttUser, mqttPass)) {
      Serial.println(" Sukses Terhubung!");
      client.subscribe(topicSubCmd);
      Serial.printf("[MQTT] Berlangganan topik kendali: %s\n", topicSubCmd);
    } else {
      Serial.printf(" Gagal (rc=%d). Mencoba lagi dalam 4 detik...\n", client.state());
      delay(4000);
    }
  }
}

void setup() {
  Serial.begin(115200);

  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW); // Inisialisasi awal: LED padam

  dht.begin();
  connectToWiFi();

  client.setServer(mqttServer, mqttPort);
  client.setCallback(onMessageCallback);
  client.setBufferSize(512);
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    connectToWiFi();
  }

  if (!client.connected()) {
```

```
    reconnectMQTT();
  }
  client.loop(); // Mendengarkan pesan kendali yang masuk secara asinkron

  // Pengiriman Telemetri Periodik Non-Blocking
  unsigned long now = millis();
  if (now - lastTeleTime >= teleInterval) {
    lastTeleTime = now;

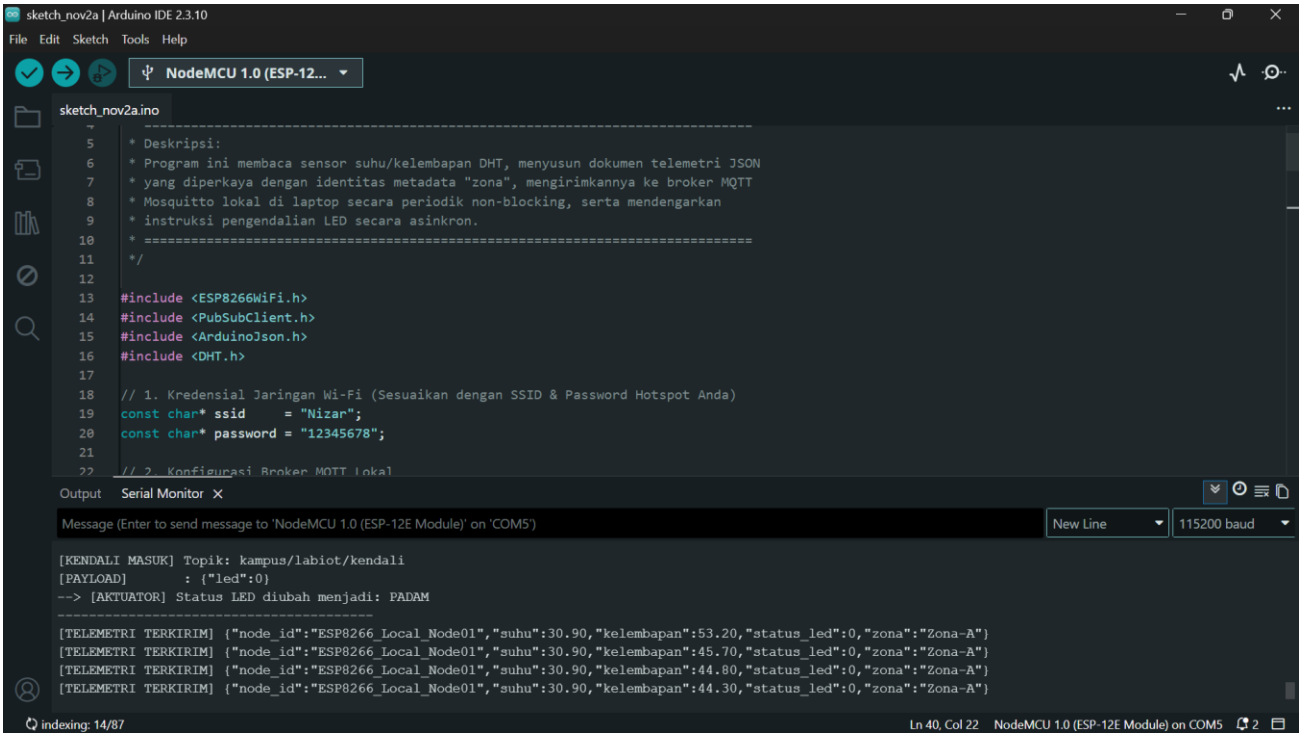
    float t = dht.readTemperature();
    float h = dht.readHumidity();

    if (!isnan(t) && !isnan(h)) {
      // Menyusun dokumen JSON telemetri
      StaticJsonDocument<256> docOut;
      docOut["node_id"]    = clientId;
      docOut["suhu"]       = serialized(String(t, 2));
      docOut["kelembapan"] = serialized(String(h, 2));
      docOut["status_led"] = digitalRead(ledPin);
      docOut["zona"]       = "Zona-A"; // Metadata penandaan zona ruang sensor

      char teleBuffer[256];
      serializeJson(docOut, teleBuffer);

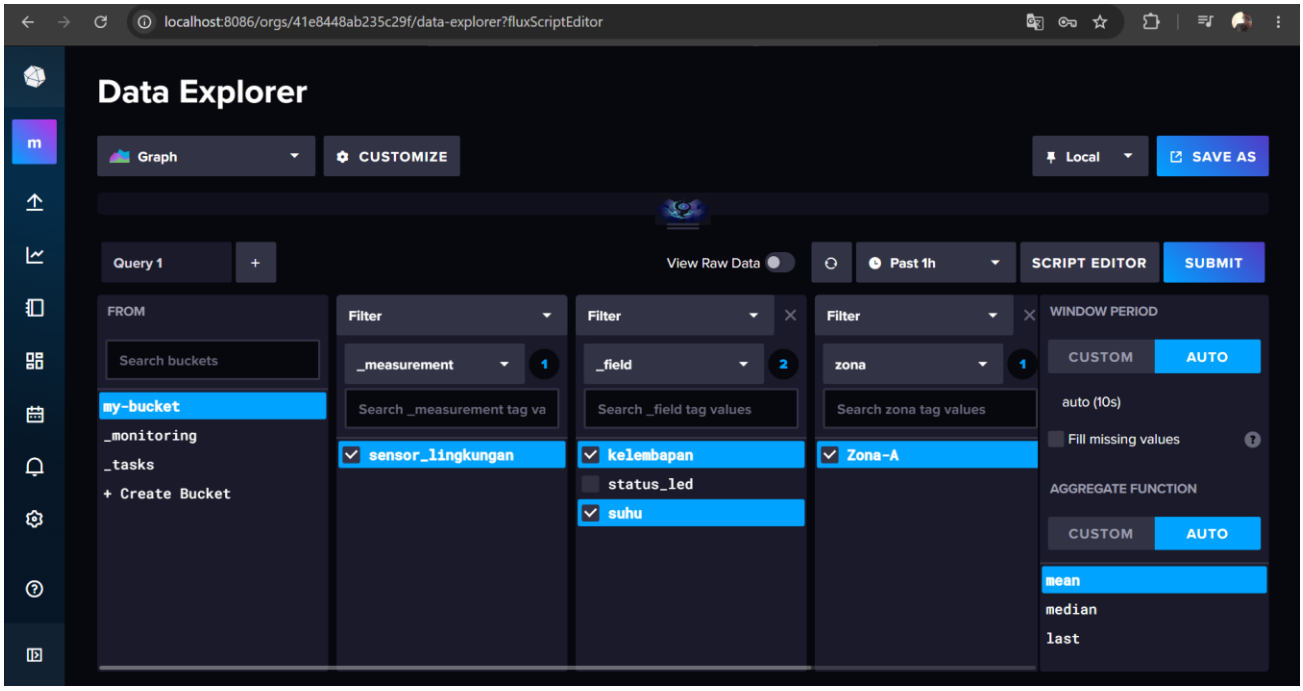
      bool success = client.publish(topicPubTele, teleBuffer);
      if (success) {
        Serial.printf("[TELEMETRI TERKIRIM] %s\n", teleBuffer);
      } else {
        Serial.println("[ERROR] Gagal memublikasikan telemetri ke broker!");
      }
    } else {
      Serial.println("[ERROR] Gagal membaca data dari sensor fisik DHT!");
    }
  }
}
```

2. Unggah (*upload*) kode program ke modul NodeMCU ESP8266.
3. Buka jendela **Serial Monitor** pada kecepatan 115200 baud. Pastikan mikrokontroler berhasil terhubung ke Wi-Fi dan broker MQTT lokal, serta mulai memublikasikan payload JSON telemetri setiap 4 detik.

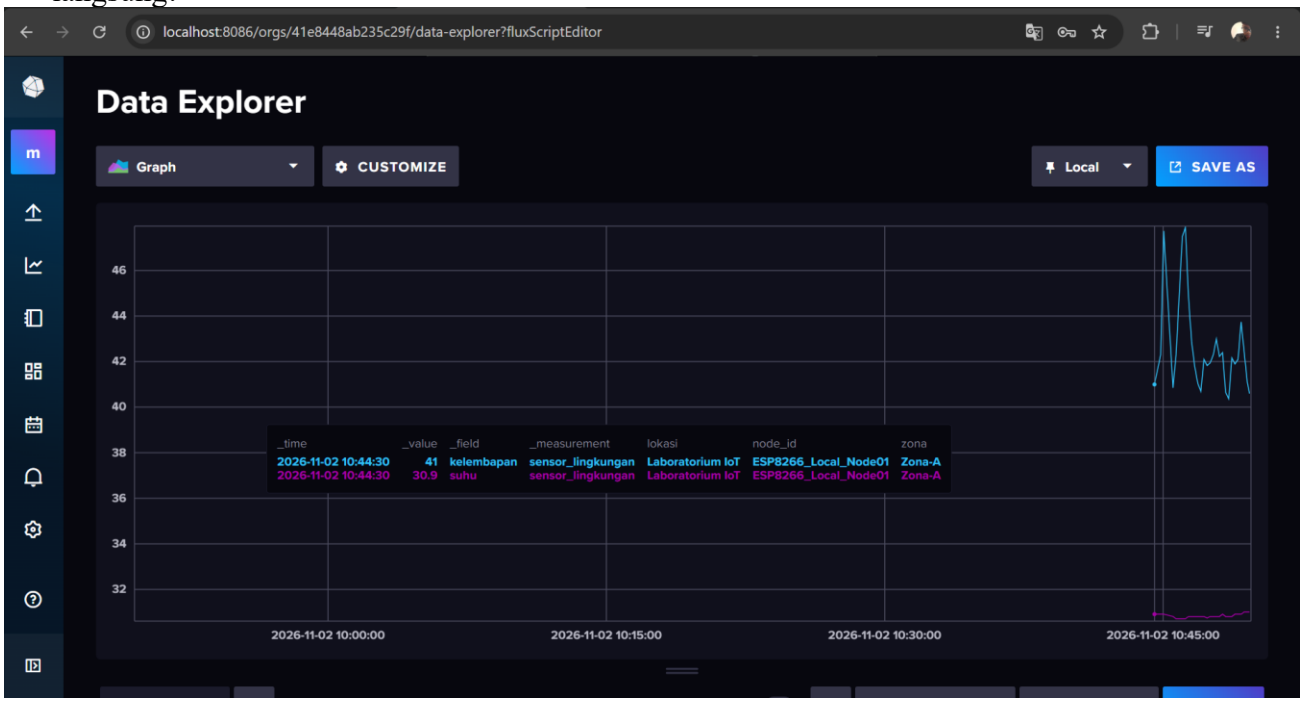


Verifikasi dan Analisis Data pada InfluxDB Data Explorer:

1. Buka kembali peramban web dan akses dashboard InfluxDB di **http://localhost:8086**.
2. Klik ikon grafik garis (**Data Explorer**) pada bilah menu navigasi di sisi kiri.
3. Di bagian bawah antarmuka, Anda akan melihat panel pembuat kueri visual (*Query Builder*). Buat penyaringan (*filter*) data secara bertahap:
    - **FROM:** Pilih ember **my-bucket**.
    - **Filter Pertama:** Kolom _measurement →beri tanda centang pada **sensor_lingkungan**.
    - **Filter Kedua:** Kolom _field →beri tanda centang pada metrik **suhu** dan **kelembapan**.
    - **Filter Ketiga (Metadata Tags):** Perhatikan kolom node_id dan zona yang muncul secara otomatis. Anda dapat membiarkannya tercentang semua atau memilih spesifik Zona-A.
4. Di sebelah kanan atas panel query builder, perhatikan opsi **Window Period** (pilih auto atau 10s) dan fungsi agregasi (pilih mean).
5. Klik tombol ungu besar bertuliskan **SUBMIT** di sebelah kanan.

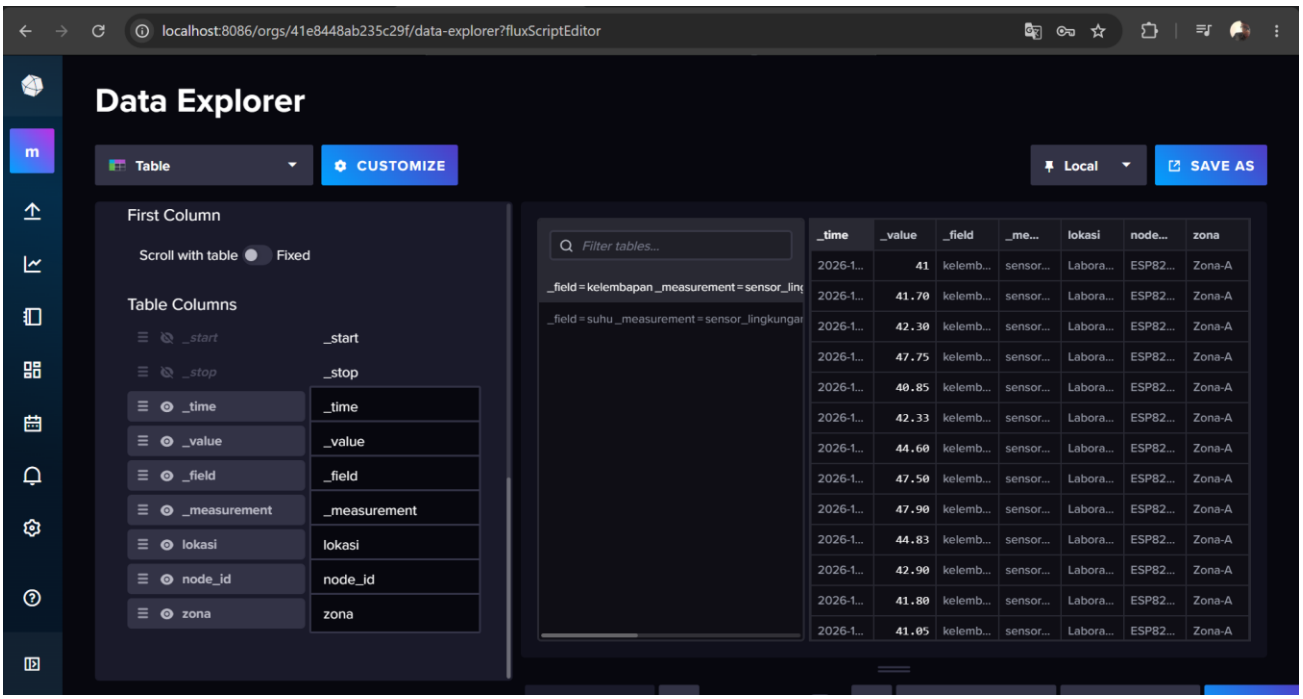


6. **Eksplorasi Representasi Data:**
   - **Tampilan Grafik Garis (*Graph View*):** Anda akan melihat grafik deret waktu interaktif yang melacak fluktuasi suhu dan kelembapan secara *real-time*. Cobalah dekatkan ujung jari Anda ke sensor DHT untuk melihat lonjakan grafik secara langsung!

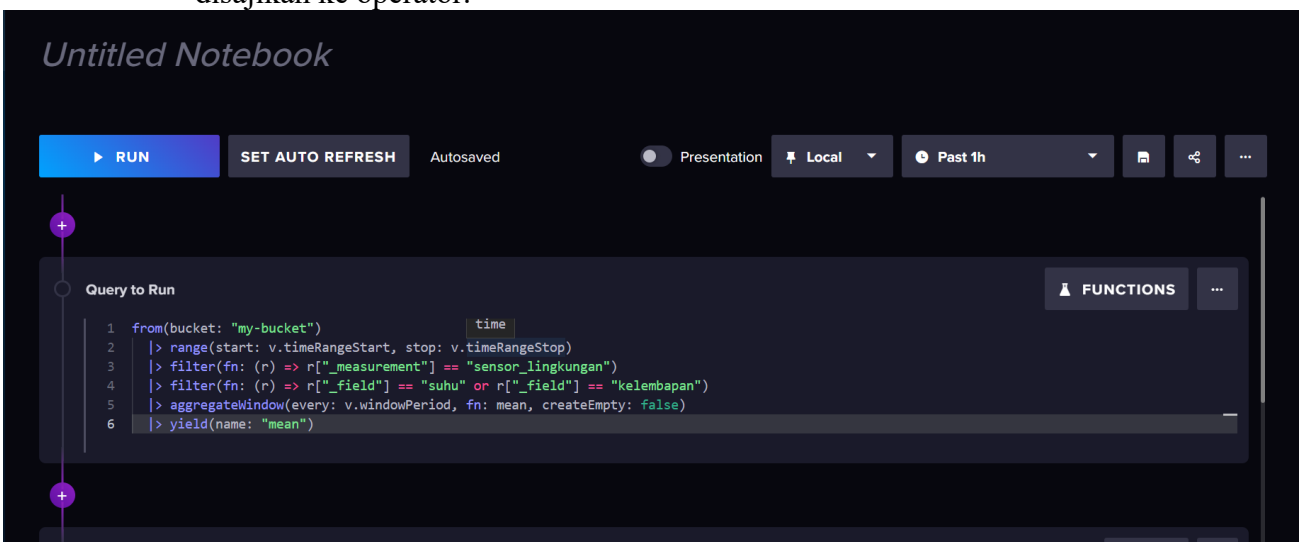


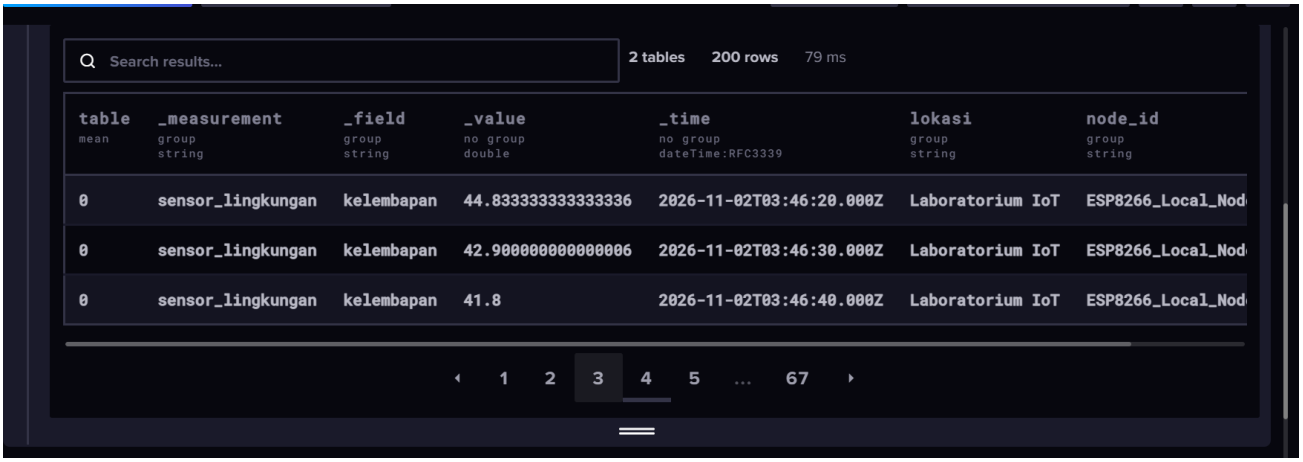
   - **Tampilan Tabel (*Table View*):** Ubah tipe visualisasi dari menu drop-down di pojok kiri atas panel grafik dari **Graph** menjadi **Table View**. Amati struktur kolom internal InfluxDB:
     - _time: Cap waktu presisi nanodetik ISO 8601.
     - _measurement: sensor_lingkungan.
     - _field: Menunjukkan baris metrik (suhu atau kelembapan).
     - _value: Nilai numerik hasil pengukuran sensor.

- node_id, lokasi, zona: Kolom metadata terindeks yang menyertai setiap baris data.



7. **Memeriksa Kode Flux di Script Editor:**
   - Klik tombol **Script Editor** di pojok kanan atas area Query Builder.
   - Amati bagaimana pilihan visual yang Anda klik tadi diterjemahkan secara otomatis oleh InfluxDB ke dalam bahasa skrip fungsional **Flux**:
   - from(bucket: "my-bucket")
   - |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
   - |> filter(fn: (r) => r["_measurement"] == "sensor_lingkungan")
   - |> filter(fn: (r) => r["_field"] == "suhu" or r["_field"] == "kelembapan")
   - |> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty: false)
   - |> yield(name: "mean")
   - Melalui antarmuka Script Editor ini, para insinyur IoT dapat menambahkan kalkulasi matematika tingkat lanjut, regresi deret waktu, dan transformasi data sebelum disajikan ke operator.




Search results...
2 tables
200 rows
79 ms
table
mean
_measurement
group
string
_field
group
string
_value
no group
double
_time
no group
dateTime:RFC3339
lokasi
group
string
node_id
group
string
0 sensor_lingkungan kelembapan 44.833333333333336 2026-11-02T03:46:20.000Z Laboratorium IoT ESP8266_Local_Nod
0 sensor_lingkungan kelembapan 42.900000000000006 2026-11-02T03:46:30.000Z Laboratorium IoT ESP8266_Local_Nod
0 sensor_lingkungan kelembapan 41.8 2026-11-02T03:46:40.000Z Laboratorium IoT ESP8266_Local_Nod
1 2 3 4 5 ... 67